

# LORQ Graphify / CRG Comparison - Final Findings

*Corrected default Graphify matrix, strict content re-judging, and prompt-time-only value analysis*

Date: 2026-06-25

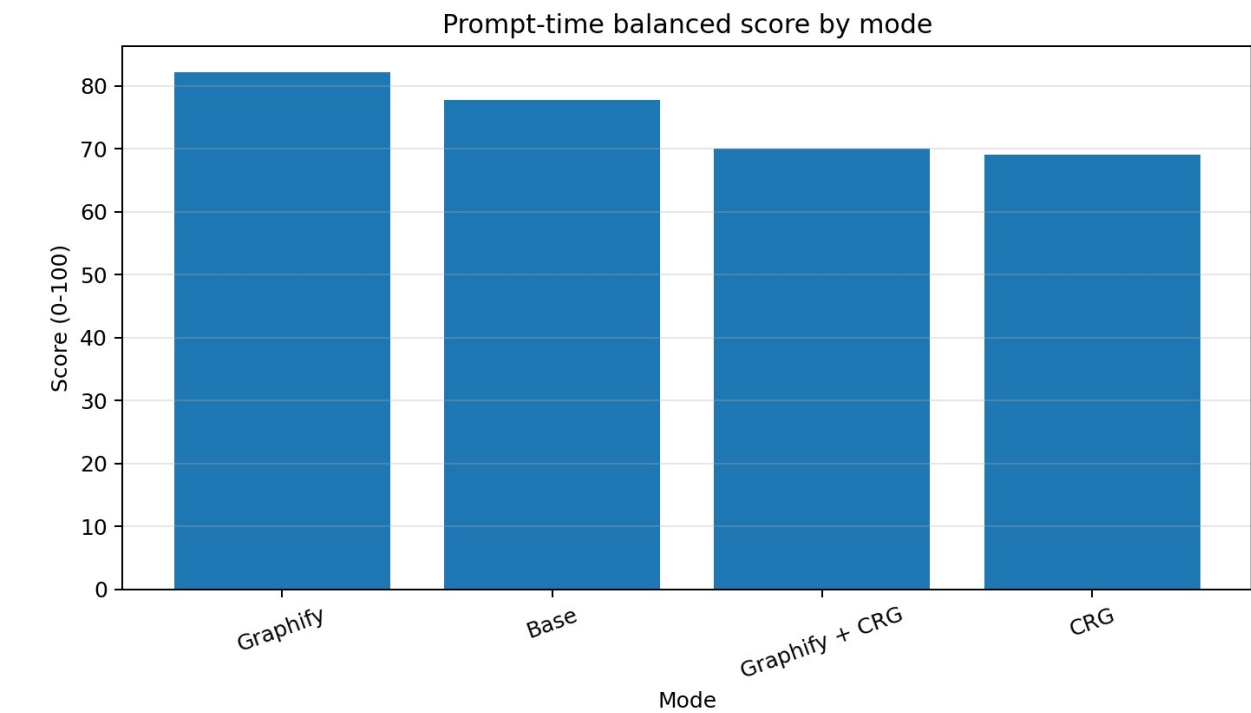
## 1. Executive summary

This artifact consolidates the full comparison of Base, Graphify, CRG, and Graphify + CRG across six nopCommerce code-understanding tasks. It supersedes the earlier Graphify-plus comparison.

- Graphify is the best steady-state default mode in the corrected run.
- Base remains a strong low-complexity fallback and wins the order-total-impact case.
- CRG-only is not compelling as an out-of-box daily default in this benchmark.
- Graphify + CRG is useful for auth-discovery but too heavy and inconsistent as a default.
- Graphify improved quality and steady-state prompt latency, but did not reduce model cost.

### Global prompt-time value summary

| Mode           | Strict content | Cost USD | Prompt time s | Setup time s (separate) | Balanced score | Quality/min |
|----------------|----------------|----------|---------------|-------------------------|----------------|-------------|
| Graphify       | 82.8           | 0.98     | 129.1         | 87.0                    | 82.2           | 39.4        |
| Base           | 79.6           | 0.91     | 145.5         | 0                       | 77.7           | 33.7        |
| Graphify + CRG | 74.1           | 1.04     | 151.8         | 108.7                   | 70.0           | 29.8        |
| CRG            | 73.1           | 0.87     | 135.2         | 24.8                    | 69.1           | 33.1        |



## 2. Analysis contract

- Matrix: 6 cases x 4 modes = 24 runs.
- Case weighting: everyday cases = 70%, hard graph-friendly cases = 30%.
- Judging: strict, presentation-grade qualitative scoring.
- Balanced score uses prompt execution time only; setup time is reported separately.
- Deterministic validation is diagnostic, not the quality score.

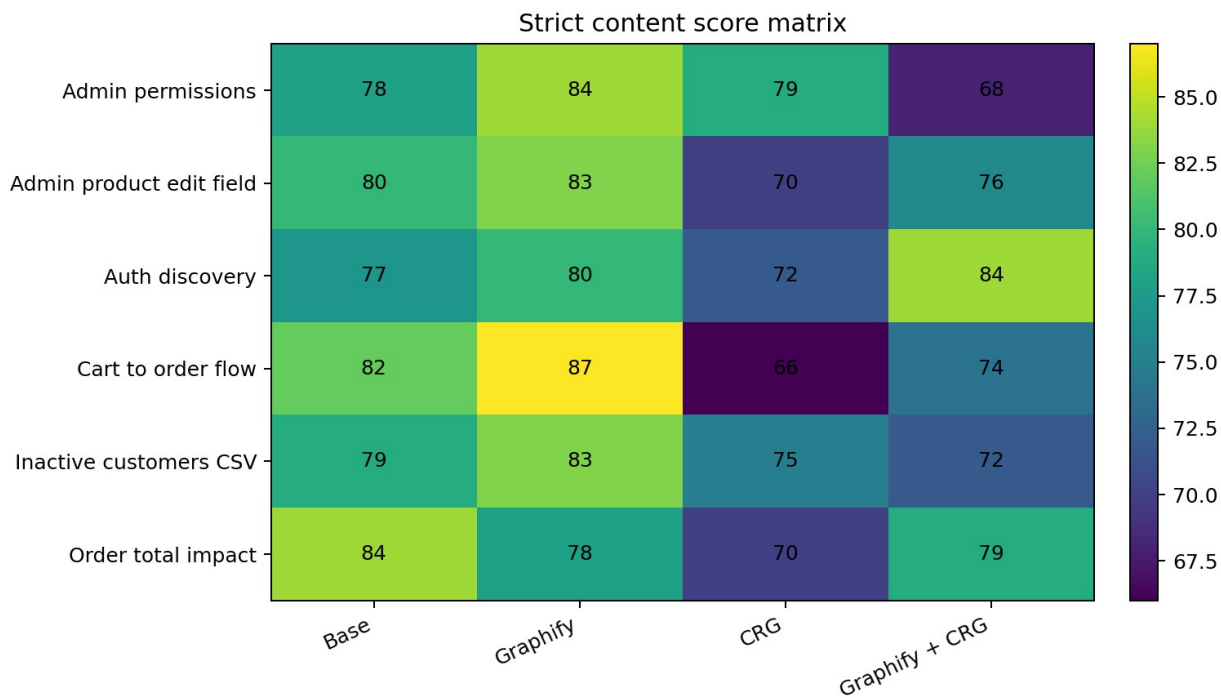
## 3. Per-case winners

| Case                     | Class               | Quality winner | Quality score | Value winner   | Value score | Cost winner | Cost USD | Prompt speed winner | Prompt time s |
|--------------------------|---------------------|----------------|---------------|----------------|-------------|-------------|----------|---------------------|---------------|
| Admin permissions        | Everyday            | Graphify       | 84            | Graphify       | 83.7        | Base        | 0.94     | Graphify            | 128.9         |
| Admin product edit field | Everyday            | Graphify       | 83            | Graphify       | 82.5        | CRG         | 0.73     | CRG                 | 109.4         |
| Auth discovery           | Hard graph-friendly | Graphify + CRG | 84            | Graphify + CRG | 82.8        | Base        | 0.8      | CRG                 | 114.7         |
| Cart to order flow       | Hard graph-friendly | Graphify       | 87            | Graphify       | 86.9        | Base        | 0.86     | Graphify            | 125.8         |
| Inactive customers CSV   | Everyday            | Graphify       | 83            | Graphify       | 82.7        | CRG         | 0.62     | Graphify            | 101.9         |
| Order total impact       | Hard graph-friendly | Base           | 84            | Base           | 83.5        | Graphify    | 0.87     | Graphify + CRG      | 157.3         |

## 4. Score matrices

### Strict content score matrix

| Case                     | base | crg | graphify | graphify+crg |
|--------------------------|------|-----|----------|--------------|
| admin-permissions        | 78   | 79  | 84       | 68           |
| admin-product-edit-field | 80   | 70  | 83       | 76           |
| auth-discovery           | 77   | 72  | 80       | 84           |
| cart-to-order-flow       | 82   | 66  | 87       | 74           |
| inactive-customers-csv   | 79   | 75  | 83       | 72           |
| order-total-impact       | 84   | 70  | 78       | 79           |



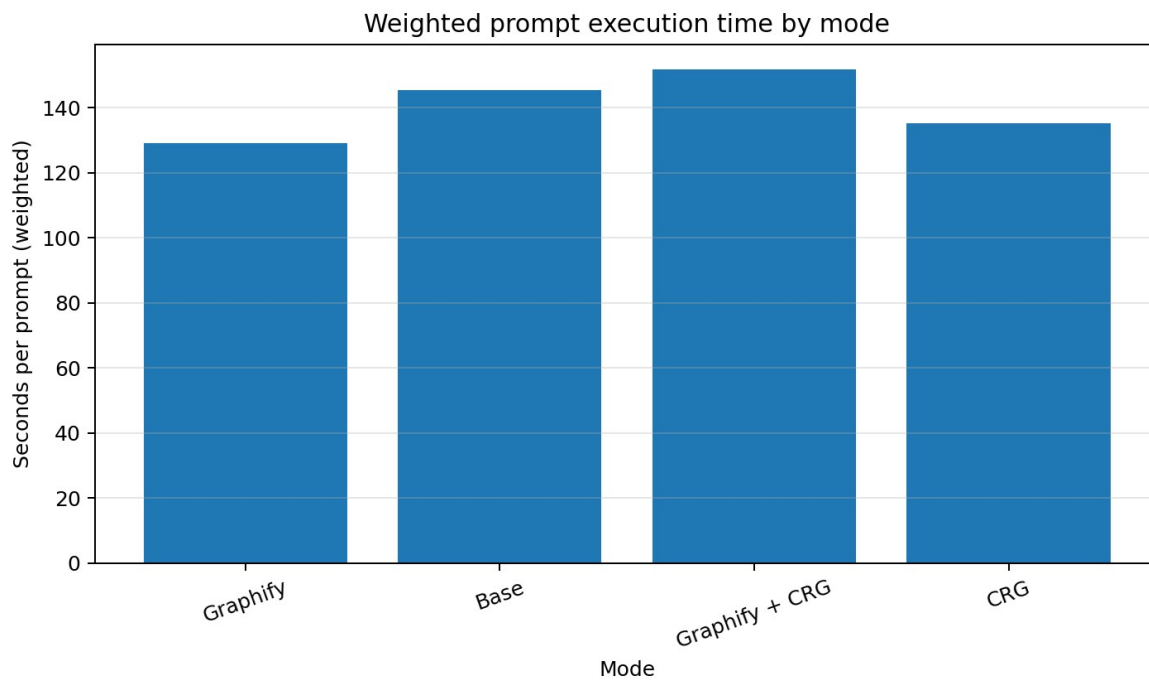
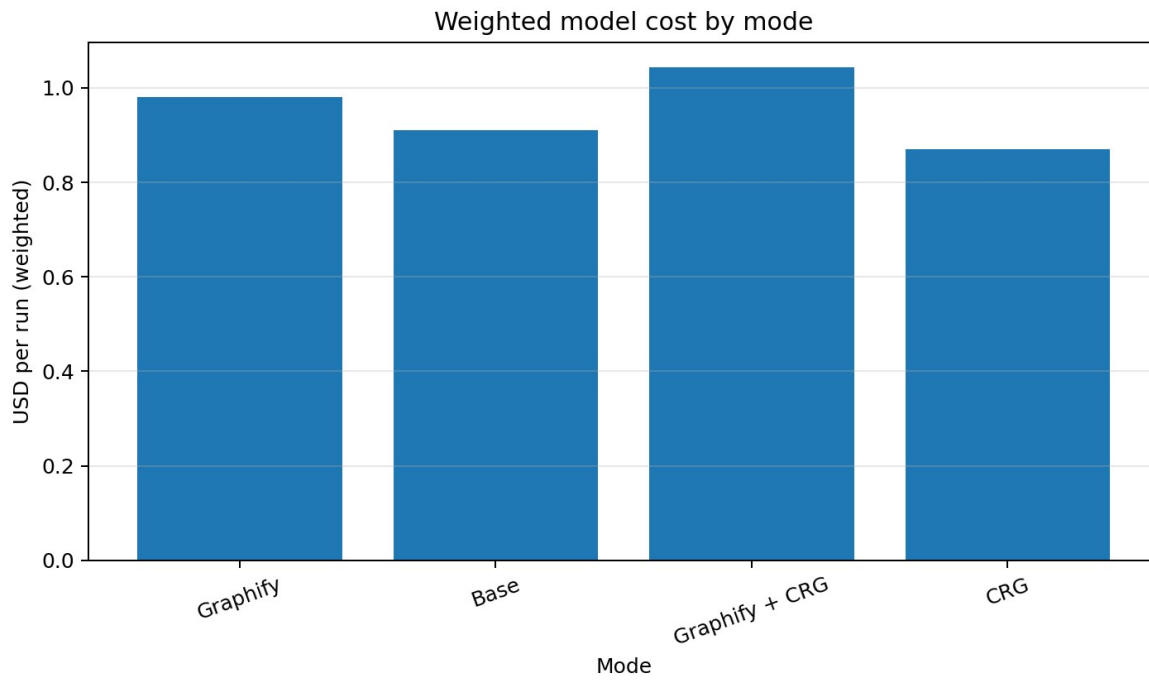
### Prompt-time balanced score matrix

| Case                     | base | crg  | graphify | graphify+crg |
|--------------------------|------|------|----------|--------------|
| admin-permissions        | 75.7 | 76.9 | 83.7     | 62.0         |
| admin-product-edit-field | 78.8 | 65.4 | 82.5     | 73.3         |
| auth-discovery           | 74.4 | 67.8 | 78.2     | 82.9         |
| cart-to-order-flow       | 80.2 | 58.5 | 86.9     | 69.1         |
| inactive-customers-csv   | 76.6 | 71.9 | 82.7     | 66.7         |
| order-total-impact       | 83.5 | 64.5 | 75.8     | 77.0         |

## 5. Cost, token, and time findings

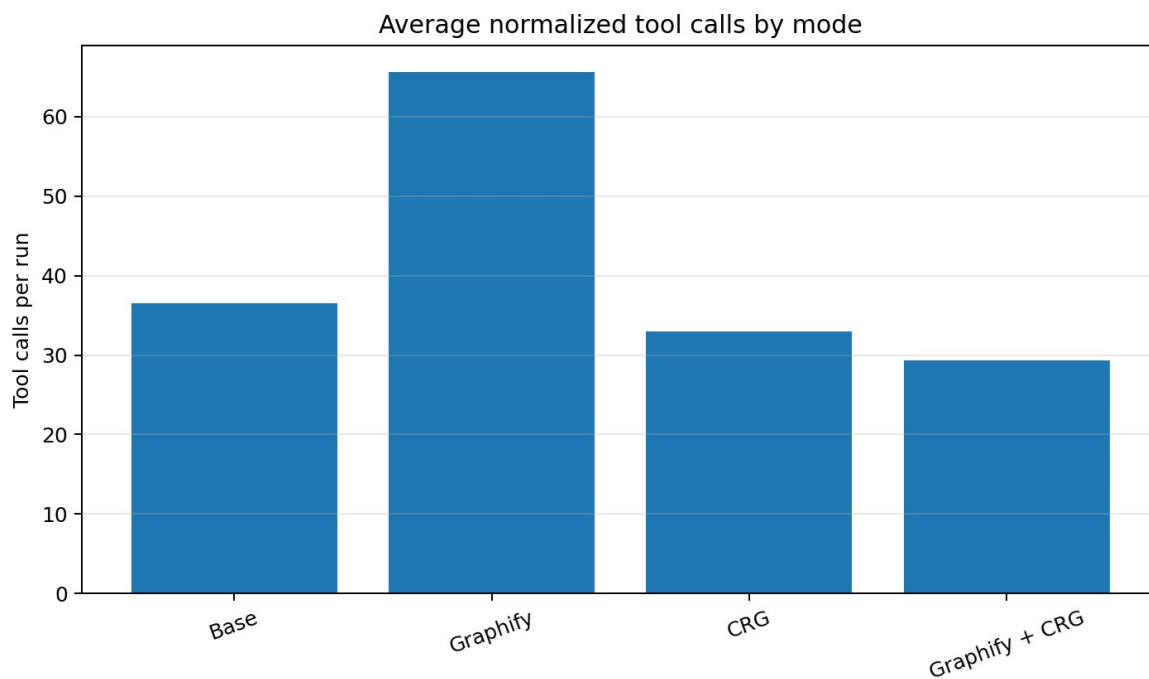
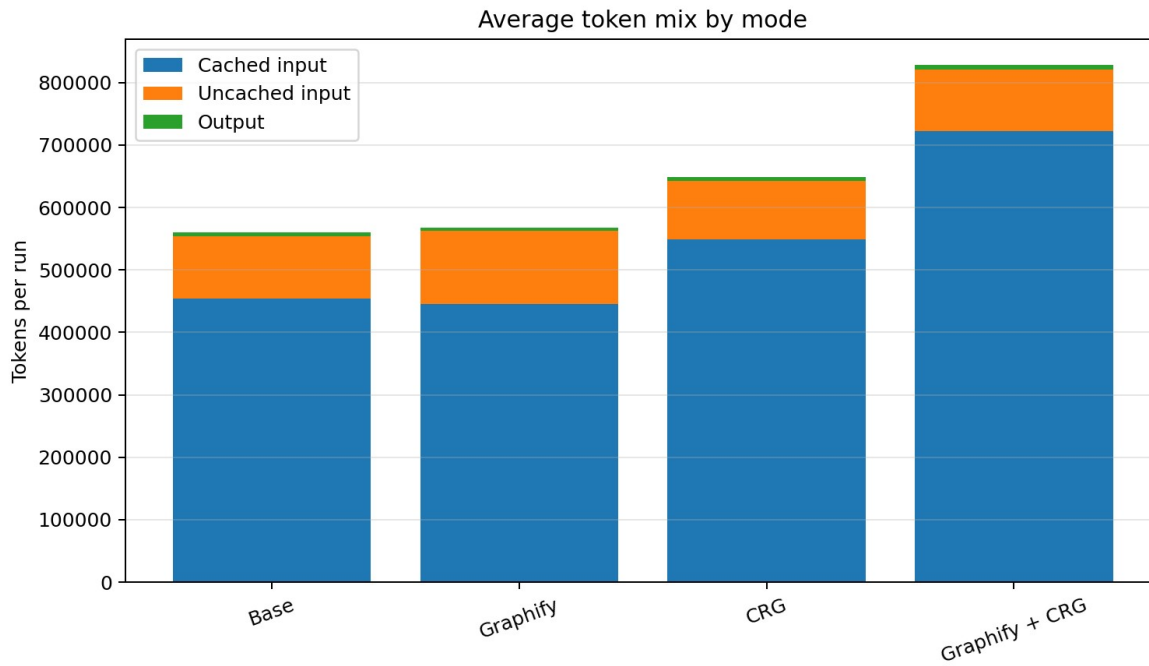
### 5.1 Cost and prompt time

| Mode           | Cost USD | Prompt time s | Setup time s (separate) | Balanced score |
|----------------|----------|---------------|-------------------------|----------------|
| Graphify       | 0.981    | 129.1         | 87.0                    | 82.1           |
| Base           | 0.91     | 145.5         | 0                       | 77.7           |
| Graphify + CRG | 1.044    | 151.8         | 108.7                   | 70.0           |
| CRG            | 0.869    | 135.2         | 24.8                    | 69.1           |



## 5.2 Token and tool summary

| Mode           | Input    | Cached input | Uncached input | Output | Reasoning output | Total tokens | Avg cost USD | Prompt time s | Setup time s | Tool calls | CRG calls | Graphify calls | Cache hit rate | Answer words |
|----------------|----------|--------------|----------------|--------|------------------|--------------|--------------|---------------|--------------|------------|-----------|----------------|----------------|--------------|
| Base           | 554389.5 | 453717.3     | 100672.2       | 6044.7 | 1218.8           | 560434.2     | 0.91         | 147.8         | 0            | 36.5       | 0         | 0              | 0.81           | 969.5        |
| CRG            | 643125.3 | 549120.0     | 94005.3        | 5425.2 | 1089.2           | 648550.5     | 0.91         | 137.5         | 24.8         | 33.0       | 4.33      | 0              | 0.85           | 884.2        |
| Graphify       | 562340.5 | 445653.3     | 116687.2       | 5414.5 | 455.3            | 567755.0     | 0.97         | 133.6         | 86.5         | 65.7       | 0         | 10.3           | 0.79           | 814.2        |
| Graphify + CRG | 821598.0 | 722986.7     | 98611.3        | 6386.3 | 1693.7           | 827984.3     | 1.05         | 157.6         | 108.7        | 29.3       | 0.33      | 6.67           | 0.88           | 905.0        |



### 5.3 Graphify cost surprise

Graphify did not reduce model cost in this run. It improved strict content quality and prompt-time value, but it had higher uncached input than Base. The result is better answers, not cheaper prompts.

- Base average uncached input: about 101k tokens per run.
- Graphify average uncached input: about 117k tokens per run.
- Graphify output was slightly lower than Base, but not enough to offset higher uncached input.
- Graphify likely improved navigation but still required source reads for evidence.

## 6. Tool usage findings

### CRG MCP calls by case

| Case                     | base | graphify | crg | graphify+crg |
|--------------------------|------|----------|-----|--------------|
| admin-product-edit-field | 0    | 0        | 6   | 0            |
| admin-permissions        | 0    | 0        | 2   | 2            |
| inactive-customers-csv   | 0    | 0        | 6   | 0            |
| auth-discovery           | 0    | 0        | 6   | 0            |
| order-total-impact       | 0    | 0        | 6   | 0            |
| cart-to-order-flow       | 0    | 0        | 0   | 0            |

### Normalized tool calls by case

| Case                     | base | graphify | crg | graphify+crg |
|--------------------------|------|----------|-----|--------------|
| admin-product-edit-field | 28   | 58       | 38  | 26           |
| admin-permissions        | 26   | 70       | 34  | 28           |
| inactive-customers-csv   | 32   | 64       | 22  | 30           |
| auth-discovery           | 61   | 74       | 32  | 32           |
| order-total-impact       | 38   | 64       | 42  | 30           |
| cart-to-order-flow       | 34   | 64       | 30  | 30           |

- Graphify + CRG made only 2 total CRG MCP calls across six combined-mode runs.
- CRG-only made no CRG MCP calls on cart-to-order-flow, the most flow-oriented case.
- Graphify used more tool calls than Base, which partly explains the lack of token savings.

## 7. Mode-by-mode interpretation

### Graphify

Role: Best default steady-state workflow

Strengths:

- Best global strict content score.
- Best prompt-time balanced score.
- Wins three everyday cases plus cart-to-order-flow.
- Improves navigation and developer handoff.

Weaknesses:

- Does not reduce model cost globally.
- Uses more tool calls and more uncached input than expected.
- Can encourage broader exploration than necessary.

### Base

Role: Strong low-complexity fallback

Strengths:

- No setup cost.
- Strong quality for plain code navigation.
- Wins order-total-impact.
- Cheaper than Graphify on weighted cost.

Weaknesses:

- Slightly weaker global strict quality.
- More prompt execution time than Graphify under steady-state scoring.
- Can be dense and broad.

### CRG-only

Role: Not recommended as daily default from this run

Strengths:

- Lowest weighted cost.
- Moderate setup time.
- Sometimes finds useful relationship context.

Weaknesses:

- Lower strict content than Base and Graphify.
- No clear out-of-box advantage.
- Weak on cart-to-order-flow.
- Tool usage inconsistent.

### Graphify + CRG

Role: Selective hard-discovery escalation

Strengths:

- Wins auth-discovery.
- Can produce broad discovery maps.

Weaknesses:

- Highest weighted cost.
- Slowest weighted prompt execution.
- Often did not use CRG MCP tools.
- Worse than Graphify alone on everyday cases.

## 8. Case-by-case interpretation

### Admin product edit field

Winner: Graphify

Best implementation map across controller, model, factory, view, domain/persistence, validation, localization, and migrations. Base was close; CRG was thinner.

### Admin permissions

Winner: Graphify

Best permission-flow architecture, including authorization attributes, OR semantics, permission service/records, and registration flow.

### Inactive customers CSV

Winner: Graphify

Best everyday export path through controller, search model, factory/service/export manager, and inactivity semantics.

### Auth discovery

Winner: Graphify + CRG

Best broad map across login, registration, password recovery, MFA, external auth, routes, views, and services. This is the main case where combined mode looked justified.

### Order total impact

Winner: Base

Most directly useful impact-analysis answer: direct consumer table, contract/service/plugin/test surfaces, and useful prioritization.

### Cart to order flow

Winner: Graphify

Best strict answer overall: clear sequence from add-to-cart through checkout/order creation and ShoppingCartItem-to-OrderItem conversion.

## 9. Caveats and recommended follow-ups

- One repetition per cell; results are directional rather than statistically definitive.
- Single repository: nopCommerce.
- LLM judging is strict but still subjective.
- Tool usage depends on agent choices; available tools can be underused.
- Graph setup was repeated for sandboxing; real-life setup economics differ.
- Graphify token savings were not shown and need separate tool-budget experiments.

Recommended follow-up experiments:

- Repeat the close cells: Base vs Graphify on everyday cases and order-total-impact.
- Test Graphify with a source-read budget to see whether it can become a token saver.
- Run CRG with verified tool usage on flow/impact tasks.
- Repeat on additional repositories.
- Run 3+ repetitions for key cells before making strong product claims.



## Appendix - strict notes by run

### admin-product-edit-field

#### graphify — 83.0

Best strict answer for this case: complete and practical, but still not a near-perfect artifact because it is broad and caveats are implicit.

**Strengths:** Very complete MVC/admin-field map with good file evidence and supporting implementation path.

**Weaknesses:** Over-complete for the prompt; could be tighter and explicitly mark optional vs mandatory files.

#### base — 80.0

Strong implementation map, but stricter grading penalizes breadth, shortened paths, and lack of crisp source-verified next-step sequencing.

**Strengths:** Covers controller/model/factory/view/domain/persistence path and gives usable implementation guidance.

**Weaknesses:** Too broad for candidate-path task; deterministic source path formatting issues; not enough separation between confirmed files and inferred follow-ups.

#### graphify+crg — 76.0

Good but not efficient or crisp: it maps the area well, but strict grading penalizes over-breadth and some plausible-but-not-proven migration/version guidance.

**Strengths:** Broad end-to-end coverage with controller/model/factory/view/persistence guidance.

**Weaknesses:** Too exhaustive; some guidance reads inferred rather than source-demonstrated; not better than Graphify alone.

#### crg — 70.0

Usable but clearly weaker than base/Graphify under stricter grading: it covers the main path, but with thinner implementation specificity.

**Strengths:** Finds required files and most adjacent components.

**Weaknesses:** Less concrete on existing field examples, UI binding, and exact implementation sequence; weaker developer handoff.

### admin-permissions

#### graphify — 84.0

Strongest strict answer for permissions: clear architecture and permission-flow evidence, though still not flawless as a step-by-step change guide.

**Strengths:** Excellent coverage of authorization attributes, OR semantics, permission service/records, and registration flow.

**Weaknesses:** Less explicit on admin UI management and concrete add-new-permission checklist.

#### crg — 79.0

Very usable but not presentation-perfect: broad structural coverage, but less crisp as a workflow handoff.

**Strengths:** Broad permission structure, plugin/admin notes, authorization behavior, service details.

**Weaknesses:** Verbose; weaker concrete “do this next” guidance; risks overwhelming a daily-workflow user.

#### base — 78.0

Solid answer, but stricter grading lowers it because it is explanatory rather than a concise implementation-ready path.

**Strengths:** Explains admin authorization layers and key permission components.

**Weaknesses:** Could better prioritize exact files/actions for adding/checking a permission; somewhat verbose.

#### graphify+crg — 68.0

Acceptable but noticeably behind: main layers are present, but it lacks enough concrete examples and implementation guidance for a high score.

**Strengths:** Covers attributes, service, records, startup, and management areas.

**Weaknesses:** Too generic in places; less evidence and less actionable than other modes.

### inactive-customers-csv

#### graphify — 83.0

Best strict answer: gives a strong everyday export implementation path with useful nuance, but still leaves a few UI/localization details underdeveloped.

**Strengths:** Clear path through controller, search model, factory/service/export manager; good inactive-definition discussion.

**Weaknesses:** Could be more explicit on button placement/localization and exact CSV formatter reuse.

#### base — 79.0

Strong and usable, but strict scoring penalizes length and incomplete UI/localization specificity.

**Strengths:** Good controller/search model/service/export path; useful inactivity semantics.

**Weaknesses:** Long; should more clearly distinguish query/filter/export/UI changes and exact CSV precedent.

#### crg — 75.0

Usable but not as strong as base/Graphify: good semantics, thinner controller/UI details.

**Strengths:** Good inactivity semantics and service/export coverage.

**Weaknesses:** Less complete on UI binding, controller action details, and implementation sequence.

#### graphify+crg — 72.0

Usable concise map, but strict grading penalizes thinner source evidence and less complete action detail.

**Strengths:** Good integration-point table and service/export/resource mentions.

**Weaknesses:** Less exhaustive evidence; less concrete about existing export action and where exactly to add UI.

### auth-discovery

#### graphify+crg — 84.0

Best strict answer for auth: complete and well grouped, though still long and with some broad/glob references.

**Strengths:** Strong workflow grouping across controller/service/MFA/external auth/routes/views.

**Weaknesses:** Long; a few view references are broad rather than exact; not all uncertainty is explicit.

#### graphify — 80.0

Strong auth discovery answer with broad coverage and better completeness, but still a large map rather than a concise workflow-level synthesis.

**Strengths:** Covers primary auth flows, external auth, MFA, services, models, routes/views.

**Weaknesses:** Large and somewhat undifferentiated; caveats and source-confirmed vs inferred boundaries could be clearer.

#### base — 77.0

Good broad auth map, but strict scoring penalizes candidate-map format and limited workflow narrative/uncertainty separation.

**Strengths:** Covers login, registration, password recovery, MFA, external auth and important files.

**Weaknesses:** More of a file inventory than a verified flow map; long and not sharply prioritized.

#### crg — 72.0

Usable source-verified map, but under strict grading it is less precise and more directory-level than the leaders.

**Strengths:** Includes core controller/service and several auth-related areas.

**Weaknesses:** Some references are broad or directory-level; less polished organization and lower precision.

### order-total-impact

#### base — 84.0

Best strict answer for impact: comprehensive and directly useful, though length keeps it below exceptional.

**Strengths:** Direct consumer table, contract/implementation/service/plugin/test surface, useful prioritization.

**Weaknesses:** Long and dense; some shortened display paths; could separate highest-risk changes more sharply.

#### graphify+crg — 79.0

Strong broad map but not clearly better than base; strict grading penalizes length and unnecessary tool/method note.

**Strengths:** Good contract/order-processing/checkout/payment/discount/tax/test coverage.

**Weaknesses:** Long; Graphify note is irrelevant; not clearly more actionable than base.

#### graphify — 78.0

Good impact map, but strict scoring penalizes density, weaker prioritization, and limited caveats.

**Strengths:** Covers contract, DI, order processing, checkout/payment/discount/tax/plugins/tests.

**Weaknesses:** Dense affected-surface list; not as risk-prioritized as base; limited caveats.

#### crg — 70.0

Usable but not strong enough: relationship evidence is useful, but prioritization and direct-consumer framing lag behind.

**Strengths:** Good relationship-style coverage of checkout/payment/plugins/tax/discount/tests.

**Weaknesses:** Verbose; less structured; misses useful direct-consumer framing and priority ranking.

## cart-to-order-flow

### graphify — 87.0

Best strict answer overall: clear source-backed cart-to-order sequence, excellent transition point, still not perfect because caveats are limited.

**Strengths:** Clear sequence from add-to-cart through checkout/order creation; strong ShoppingCartItem-to-OrderItem conversion explanation.

**Weaknesses:** Could add more explicit uncertainty/caveat notes and distinguish core vs side-effect paths.

### base — 82.0

Strong flow trace with detailed source evidence, but strict grading penalizes density and lack of concise executive sequence.

**Strengths:** Detailed chain from route/controller to cart service, checkout, PlaceOrderAsync, post-processing and cleanup.

**Weaknesses:** Long and dense; could better isolate the minimal flow from supporting detail.

### graphify+crg — 74.0

Solid but not top-tier: covers the required flow, but lacks richness and precision compared with base/Graphify.

**Strengths:** Good high-level sequence through add-to-cart, checkout, PlaceOrderAsync, SaveOrderDetailsAsync, cleanup.

**Weaknesses:** Less detail on routes, payment request, post-processing, events, persistence; not enough evidence of combined-tool value.

### crg — 66.0

Weakest strict flow answer: it is usable, but the CRG advantage did not materialize and the answer itself is less complete.

**Strengths:** Covers main required files and broad flow stages.

**Weaknesses:** Less route/post-processing/persistence detail; final note about graph MCP cancellation undermines the run; weaker evidence depth.